

2. Zahlensysteme, Codes und Gatter

2.1 Zahlensysteme

Ein Additionssystem ist ein Zahlensystem, bei dem sich der Wert einer Zahl durch Addieren der Werte ihrer Ziffern errechnet. Im Gegensatz dazu spielt bei einem Stellenwertsystem auch die Position der Ziffern eine Rolle.

Ein einfaches Beispiel für ein Additionssystem ist die Strichliste, ein Unärsystem: Hier gibt es nur eine Ziffer, beispielsweise den vertikalen Strich "|". Eine Zahl wird als Folge von Strichen dargestellt, wobei der Wert einer Zahl der Anzahl der Striche entspricht. Die dezimale Zahl 3 zum Beispiel wird in diesem System als ||| geschrieben. Eine solche Schreibweise wird aber bei großen Zahlen schnell unübersichtlich, so dass die Notwendigkeit erwächst, weitere Ziffern einzuführen.

Bei den Römischen Zahlen gibt es sieben Ziffern und zwar I, V, X, L, C, D und M. Diesen entsprechen die Werte 1, 5, 10, 50, 100, 500 und 1000. Mit Ausnahme der Subtraktionsschreibweise spielt die Reihenfolge der Ziffern einer römischen Zahl keine Rolle für den Wert der Zahl, wenngleich es üblich ist, die Ziffern von links nach rechts absteigend zu ordnen. Prinzipiell sind aber die drei Zahlen XII, IXI, IIX äquivalent und entsprechen der dezimalen 12.

Bei den Stellenwertsystemen (Polyadische Zahlensysteme) hat jede Ziffer abhängig von ihrer Stellung in der Ziffernfolge einen bestimmten Wert, der aus der Stellung und der zugrunde liegenden Basis b berechnet werden kann.

Für eine beliebige Zahl (Ziffernfolge) zur Basis b mit n Vorkomma- und m Nachkommastellen gilt:

$$Z = (z_{n-1} \dots z_0, z_{-1} \dots z_{-m})_b = z_{n-1} \cdot b^{n-1} + \dots + z_0 \cdot b^0 + z_{-1} \cdot b^{-1} + \dots + z_{-m} \cdot b^{-m}$$

$$= \sum_{k=-m}^{n-1} z_k \cdot b^k \quad (\text{Gl. 2.1})$$

Beispiele für gebräuchliche Zahlensysteme:

Basis b	Name	Ziffernvorrat	Beispielzahl
2	Dualsystem	0,1	$(1000001100)_2$
3	Ternärsystem	0,1,2	$(201102)_3$
8	Oktalsystem	0 ... 7	$(1014)_8$
10	Dezimalsystem	0 ... 9	$(524)_{10}$
16	Hexadezimalsystem	0 ... 9, A ... F	$(20C)_{16}$

Ein polyadisches Zahlensystem zur Basis b hat einen Ziffernvorrat von b Ziffern, mit denen jede Stelle in der Ziffernfolge belegt sein kann. Gültige Ziffern liegen in dem Bereich 0 bis $b-1$.

Zur Kennzeichnung des verwendeten Systems (falls nicht eindeutig ersichtlich) setzt man die Ziffernfolge in Klammern und schreibt die Basis als Index.

Im Alltag üblich ist das Dezimalsystem:

$$\begin{aligned} Z &= (z_{n-1} \dots z_0, z_{-1} \dots z_{-m})_{10} \\ &= z_{n-1} \cdot 10^{n-1} + \dots + z_0 \cdot 10^0 + z_{-1} \cdot 10^{-1} + \dots + z_{-m} \cdot 10^{-m} \\ &= \sum_{k=-m}^{n-1} z_k \cdot 10^k \end{aligned}$$

Umwandlung von einem Stellenwertsystem beliebiger Basis ins Dezimalsystem:

Diese Umwandlung erfolgt durch Anwendung von Gleichung (2.1).

Beispiel:

$$(210210)_3 = 2 \cdot 3^5 + 1 \cdot 3^4 + 0 \cdot 3^3 + 2 \cdot 3^2 + 1 \cdot 3^1 + 0 \cdot 3^0 = (588)_{10} = 588$$

Umwandlung vom Dezimalsystem in ein polyadisches Zahlensystem der Basis b :

1) Umwandlung durch Division

Die grundlegende Idee kann aus der folgenden Darstellung der Zahl abgeleitet werden:

$$\begin{aligned} Z &= \sum_{s=0}^{n-1} z_s \cdot b^s \\ &= z_{n-1} \cdot b^{n-1} + z_{n-2} \cdot b^{n-2} + \dots + z_1 \cdot b^1 + z_0 \cdot b^0 \\ &= (((((z_{n-1} \cdot b + z_{n-2}) \cdot b + z_{n-3}) \cdot b + \dots + z_2) \cdot b + z_1) \cdot b + z_0) \end{aligned}$$

Dividiert man Z durch b , so ergibt sich als Divisionsrest z_0 . Das Ergebnis der ganzzahligen Division ist dann:

$$(((z_{n-1} \cdot b + z_{n-2}) \cdot b + z_{n-3}) \cdot b + \dots + z_2) \cdot b + z_1$$

Dieses Ergebnis kann man nun wieder durch b dividieren; der Divisionsrest ist z_1 .

Das Verfahren mit der fortlaufenden Division durch b wird so lange fortgesetzt, bis das Ergebnis der ganzzahligen Division 0 ist. Die Divisionsreste sind die

gesuchten Ziffern (Reihenfolge beachten: Zuerst erhält man die niederwertigste Ziffer!)

Beispiel 1: Wandle $(588)_{10}$ in das Trialsystem:

$$\begin{array}{rcl}
 588 & / & 3 = 196 \text{ Rest } 0 \\
 196 & / & 3 = 65 \text{ Rest } 1 \\
 65 & / & 3 = 21 \text{ Rest } 2 \\
 21 & / & 3 = 7 \text{ Rest } 0 \\
 7 & / & 3 = 2 \text{ Rest } 1 \\
 2 & / & 3 = \boxed{0} \text{ Rest } 2
 \end{array}$$

Ergebnis: $(588)_{10} = (210210)_3$

Beispiel 2: Wandle $(524)_{10}$ in das Dualsystem:

$$\begin{array}{rcl}
 524 : 2 = 262 & + & 0 \\
 262 : 2 = 131 & + & 0 \\
 131 : 2 = 65 & + & 1 \\
 65 : 2 = 32 & + & 1 \\
 32 : 2 = 16 & + & 0 \\
 16 : 2 = 8 & + & 0 \\
 8 : 2 = 4 & + & 0 \\
 4 : 2 = 2 & + & 0 \\
 2 : 2 = 1 & + & 0 \\
 1 : 2 = 0 & + & 1
 \end{array}$$

Leserichtung ↑

$$(524)_{10} = (1000001100)_2$$

Leserichtung →

2) Umwandlung durch Subtraktion

- Bestimme die größte Potenz b^n , die kleiner oder gleich der umzuwandelnden Zahl Z ist.
- Dividiere Z ganzzahlig durch b^n . Das Ergebnis der Division ist die erste gesuchte Ziffer z_n .
- Der Divisionsrest ist die Differenz $D_n = Z - z_n * b^n$.
Dividiere D_n durch b^{n-1} . Das Ergebnis der Division ist die Ziffer z_{n-1} .
Der Divisionsrest ist die Differenz $D_{n-1} = D_n - z_{n-1} * b^{n-1}$.
- Wiederhole dieses Verfahren bis zur Division durch b^0 .
In diesem Schritt erhält man z_0 .
Als Divisionsrest muss sich 0 ergeben.

Beispiel: Wandle $(61)_{10}$ in das Ternärsystem:

Höchste Potenz: 3^3 ; denn: $3^3 \leq 61$, $3^4 > 61$

$$\begin{array}{lcl}
 61 / 3^3 = 61 / 27 = 2 & \text{Rest } 7: & (D_3 = 61 - 2 \cdot 3^3 = 7) \\
 7 / 3^2 = 7 / 9 = 0 & \text{Rest } 7: & (D_2 = 7 - 0 \cdot 3^2 = 7) \\
 7 / 3^1 = 7 / 3 = 2 & \text{Rest } 1: & (D_1 = 7 - 2 \cdot 3^1 = 1) \\
 1 / 3^0 = 1 / 1 = 1 & \text{Rest } 0: & (D_0 = 1 - 1 \cdot 3^0 = 0)
 \end{array}$$

Ergebnis: $(61)_{10} = (2021)_3$

2.2 Dualzahlen

Für die Digitaltechnik ist in erster Linie das Dualsystem wichtig. Wegen der leichten Umwandelbarkeit spielt auch das Hexadezimalsystem (früher das Oktalsystem) eine wichtige Rolle, da damit eine kompaktere Notation möglich ist.

Bei einer Dualzahl wird die höherwertigste Stelle als MSB (most significant bit) bezeichnet, die niederwertigste Stelle als LSB (least significant bit).

Beispiel:

MSB (Most significant bit)							LSB (Least significant bit)
↓							↓
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	0	0	1	1	0	0	1

$= 1 \cdot 2^0 + 1 \cdot 2^3 + 1 \cdot 2^4 + 1 \cdot 2^7 = 153$

Umwandlung zwischen Dual- und Hexadezimalsystem:

Bei der Umwandlung vom Hexadezimal- ins Dualsystem kann man die einzelnen Ziffern separat wandeln, da die Basis 16 eine Potenz der Basis 2 ist.

Beispiel:

$$\begin{aligned}
 (A3B)_{16} &= 10 \cdot 16^2 + 3 \cdot 16^1 + 11 \cdot 16^0 \\
 &= (1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) \cdot 2^8 \{= 16^2\} \\
 &+ (0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) \cdot 2^4 \{= 16^1\} \\
 &+ (1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) \cdot 2^0 \{= 16^0\} \\
 &= \left(\underbrace{1\ 0\ 1\ 0}_A \underbrace{0\ 0\ 1\ 1}_3 \underbrace{1\ 0\ 1\ 1}_B \right)_2
 \end{aligned}$$

Allgemein gelten die folgenden Regeln:

1) Umwandlung Hexadezimalsystem $\rightarrow$ Dualsystem:

Jede Hexadezimalziffer wird in 4 Dualziffern gewandelt.

2) Umwandlung Dualsystem $\rightarrow$ Hexadezimalsystem:

Jeweils 4 Dualziffern werden rechtsbündig zusammengefasst, gegebenenfalls sind links führende Nullen zu ergänzen. Jede Gruppe von 4 Dualziffern wird in eine Hexadezimalziffer gewandelt.

Für die Wandlung der Hexadezimalziffern zu den Dualziffern gilt die folgende Tabelle:

Hex.	Dual	Hex.	Dual	Hex.	Dual	Hex.	Dual
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

Arithmetik von Dualzahlen:

Die Addition im Dualsystem erfolgt analog zum (gewohnten) Addieren im Dezimalsystem. Es gelten die Regeln:

0 mal 1 $\Rightarrow$ Ergebnis 0, kein Übertrag

1 mal 1 $\Rightarrow$ Ergebnis 1, kein Übertrag

2 mal 1 $\Rightarrow$ Ergebnis 0, Übertrag 1

3 mal 1 $\Rightarrow$ Ergebnis 1, Übertrag 1

Beispiel:

$$\begin{array}{r}
 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \\
 + \quad 0_1 \quad 1_1 \quad 1_1 \quad 0_1 \quad 1_1 \quad 1 \quad 0 \\
 \hline
 1 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 1
 \end{array}$$

Die **Multiplikation** von Dualzahlen erfolgt ebenso wie die Multiplikation von Dezimalzahlen. Die Regeln für die Multiplikation lauten:

$$0 \cdot 0 = 0$$

$$1 \cdot 0 = 0$$

$$0 \cdot 1 = 0$$

$$1 \cdot 1 = 1$$

Beispiel:

$$\begin{array}{r}
 10110 * 10110 \\
 \hline
 10110 \\
 0 \\
 10110 \\
 10110 \\
 0 \\
 \hline
 111100100
 \end{array}$$

Ein wichtiger Sonderfall ist die Multiplikation mit einer Zweierpotenz. Bei der Multiplikation der Dualzahl z mit einer Zweierpotenz 2^n werden der Zahl z genau n Nullen angehängt.

Beispiel:

$$(11001)_2 \cdot (1000)_2 = (11001000)_2$$

Entsprechendes gilt für die Division mit einer Zweierpotenz 2^n . Am Ende der Zahl werden n Nullen weggestrichen bzw. das Komma wird um n Stellen nach links verschoben.

Beispiel:

$$(11011)_2 \div (1000)_2 = (11,011)_2$$

Darstellung negativer Zahlen:

Für die Darstellung negativer Zahlen im Dualsystem gibt es mehrere Möglichkeiten. Die einfachste Möglichkeit besteht darin, das oberste Bit als Vorzeichen zu interpretieren (Vorzeichen-Betrags-Darstellung):

Vorzeichen

	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	1	0	0	1	1	0	0

$$= -1 \cdot (16 + 8 + 1) = -25$$

Vorzeichen-Betrags-Darstellung der Zahl -25

Der Nachteil dieser Darstellung liegt darin, dass die obigen Rechenregeln nicht mehr gelten und somit keine die einfache Arithmetik mehr möglich ist.

Bei der Einerkomplement-Darstellung werden alle Bits invertiert. Das MSB kann dann wieder als Vorzeichenbit interpretiert werden, ist es 1, ist die Zahl negativ und für den Zahlenwert liefern die Stellen mit 0 einen Beitrag:

Vorzeichen

↓

	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	1	1	0	0	1	1	0

$$= -1 \cdot (16 + 8 + 1) = -25$$

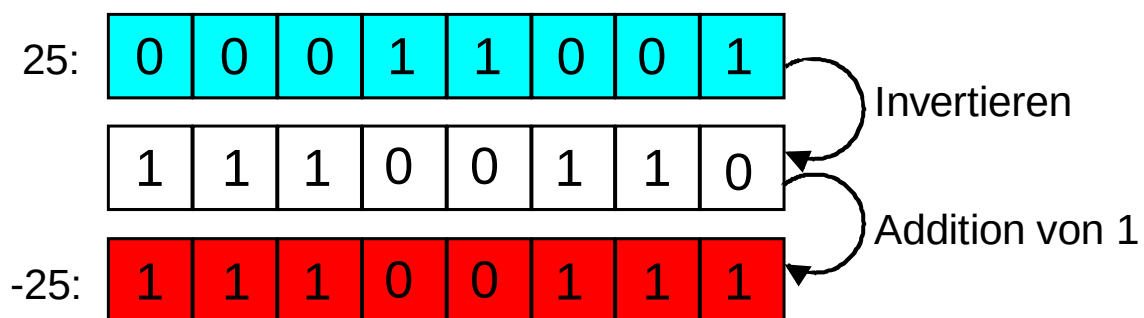
Darstellung der Zahl -25 als Einerkomplement

Zahl und Gegenzahl (d.h. gleicher Betrag, verschiedenes Vorzeichen) ergeben bei der Addition das Einswort, das eine zweite Darstellung (neben dem Nullwort) des Wertes 0 ist.

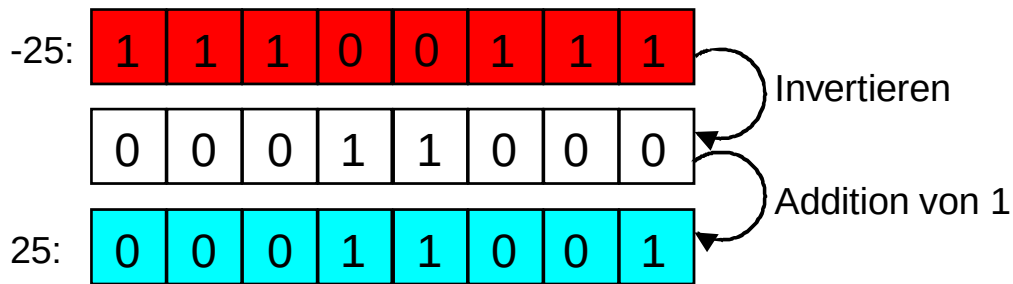
Allerdings "stimmt" die Arithmetik nicht in allen Fällen; durch die "doppelte 0" kann es vorkommen, dass das Ergebnis um 1 daneben liegt. Dies zeigen die beiden folgenden Beispiele:

7: 0111	3: 0011
-3: 1100	-7: 1000
0011 → 3 ≠ 4	1011 → -4 o.k.

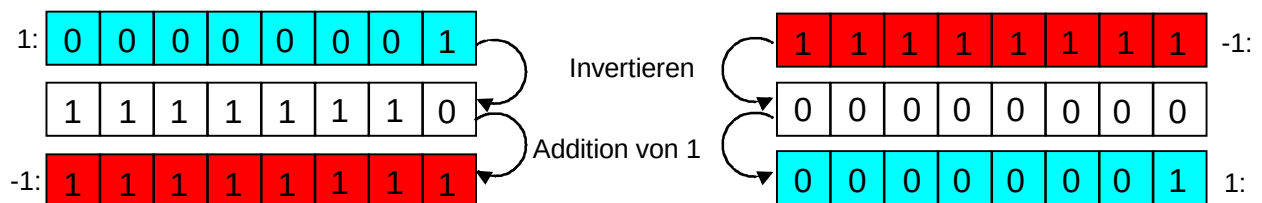
Abhilfe schafft hier die Zweierkomplement-Darstellung. Bei dieser wird nach der Inversion noch 1 addiert (ein eventueller Übertrag wird dabei nicht berücksichtigt).



Auch der umgekehrte Weg (von der negativen Zahl zur positiven Gegenzahl) erfolgt durch die Schritte „Invertieren“ und „Addition von 1“:



Die Addition von Zahl und Gegenzahl ergibt jetzt immer das Nullwort, das jetzt die einzige Darstellung der Zahl 0 ist. Das Einswort entspricht immer der Zahl -1, wie das folgende Beispiel mit 8 Bit verdeutlicht:



Die folgende Tabelle zeigt am Beispiel von 4-Bit-Zahlen, wie diese interpretiert werden können: ohne Vorzeichen (engl.: **unsigned**), als Vorzeichen-Betrags-Darstellung, als Einerkomplement-Darstellung und als Zweierkomplement-Darstellung (engl.: **signed**):

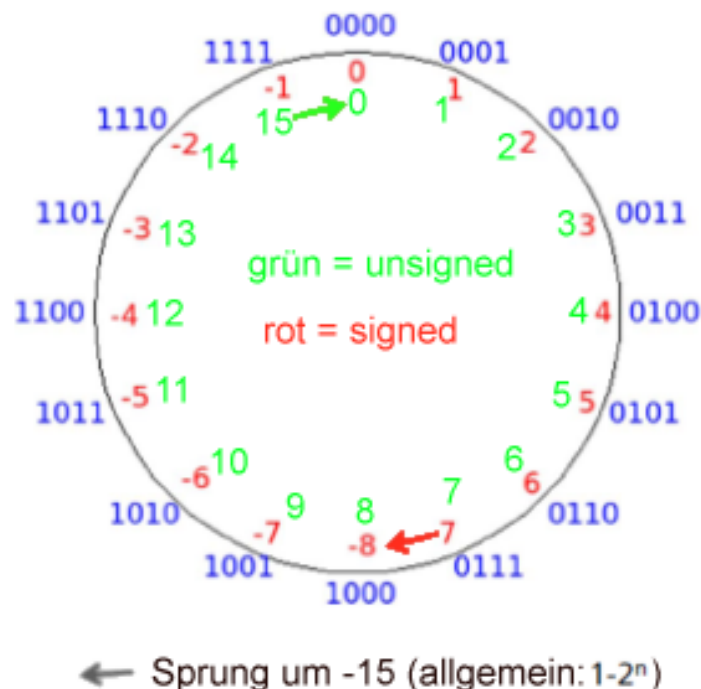
4-Bit-Zahl	ohne Vorzeichen (unsigned)	Vorzeichen / Betrag	1-er Komplement	2-er Komplement (signed)
0000	0	0	0	0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	4	4	4	4
0101	5	5	5	5
0110	6	6	6	6
0111	7	7	7	7
1000	8	-0	-7	-8
1001	9	-1	-6	-7
1010	10	-2	-5	-6
1011	11	-3	-4	-5
1100	12	-4	-3	-4
1101	13	-5	-2	-3
1110	14	-6	-1	-2
1111	15	-7	-0	-1

Die obigen Rechnungen im Zweierkomplement lauten nun:

7: 0111
 -3: 1101
 0100 → 4 o.k.

3: 0011
 -7: 1001
 1100 → -4 o.k.

Natürlich stimmen die Ergebnisse nicht mehr, wenn der Zahlenbereich überschritten wird (**overflow**). Dies ist der Fall, wenn die Rechnung über den "Sprung" zwischen der größten Zahl und der kleinsten Zahl führt. Diese Stelle ist in der folgende Grafik am Beispiel von 4-Bit-Zahlen mit dem roten Pfeil markiert:



Der darstellbare Zahlenbereich bei n Bit ist der Bereich von -2^{n-1} bis $2^{n-1}-1$. Der größte Wert wird durch die Zahl (01...1) repräsentiert, die kleinste durch (10...0).

Man kann das MSB bei der Zweierkomplementdarstellung auch als negativen Anteil auffassen. Der Wert einer n -stelligen Dualzahl berechnet sich dann wie folgt:

$$Z = -z_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} z_i \cdot 2^i$$

Man kann leicht verifizieren, dass man mit dieser Formel zu den gleichen Ergebnissen kommt wie mit den oben beschriebenen Schritten über das Einer-Komplement als Zwischenergebnis.

Die folgende Graphik veranschaulicht das am Beispiel der Zahl -25 in 8-Bit-Darstellung:

$$\begin{array}{cccccccc}
 -2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 \hline
 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\
 \hline
 = -128 + 64 + 32 + & & & & & 4 + 2 + 1 & & = -25
 \end{array}$$

Darstellung der Zahl -25 als Zweierkomplement

2.3 Binäre Codes

Grundbegriffe:

Definition (Zeichenvorrat, Alphabet):

Ein Zeichenvorrat ist eine ungeordnete Menge von Zeichen.

Ein Alphabet ist eine geordnete Menge von Zeichen („alphabetische Reihenfolge“).

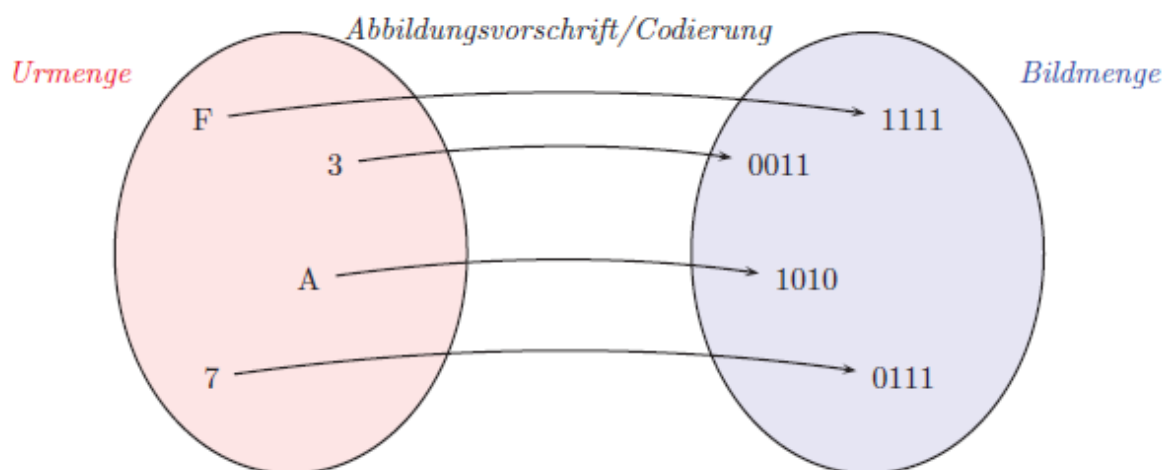
Definition (Zeichenkette):

Eine Zeichenkette ist ein geordnetes n -Tupel von Zeichen.

Bei einer Zeichenkette ist die Reihenfolge der Anordnung der Zeichen von Bedeutung.

Definition (Code nach DIN44300)

Ein Code ist eine Vorschrift für die eindeutige Zuordnung der Zeichen eines Zeichenvorrates zu denjenigen eines anderen Zeichenvorrates.



Anmerkungen:

- Die Zeichen des ursprünglichen Zeichenvorrates nennt man auch Urmenge, die des anderen Bildmenge.
- Für die Bildmenge sind auch Zeichenketten zulässig.
- Ein Code ist (mathematisch betrachtet) eine eindeutige Abbildung.

Für die Bildmenge gibt es zwei Extremfälle. Wenn der Zeichenvorrat sehr groß ist, dann hat man i.a. sehr kurze Wortlängen. Man denke hierbei an chinesische Schriftzeichen. Dieses Alphabet umfasst sehr viele verschiedene Zeichen im Gegensatz zum lateinischen Alphabet, dafür benötigt man i.a. deutlich weniger Zeichen, um denselben Sachverhalt auszudrücken. Chinesische Zeitungen sind dünner. . .

Im zweiten Extremfall hat man einen sehr kleinen Vorrat an Zeichen. Es ergeben sich entsprechend größere Wortlängen. Man denke hierbei an die Dualzahlen, die deutlich mehr Stellen benötigen als "normale" Dezimalzahlen.

Definition (Binärcode):

Ein Binärcode ist ein Code mit genau zwei Zeichen im Zeichenvorrat der Bildmenge.

Ein Beispiel für einen Binärcode ist das Morsealphabet (bei Vernachlässigung der „Abstandspause“). Die beiden Zeichen sind "·" (Punkt) und "–" (Strich) und es gilt z.B.

<i>a</i>	⇒	· –
<i>b</i>	⇒	– ...
<i>e</i>	⇒	·
<i>r</i>	⇒	· – ·

Wenn alle Wortlängen eines Codes gleich lang sind, so spricht man von einem Blockcode.

Definition (Blockcode):

Ein Blockcode ist ein Code, dessen Wortlänge immer gleich groß ist.

Ansonsten spricht man von einem Code beliebiger Länge, wozu auch der Morsecode gehört.

Eine andere Kategorisierung von Codes unterscheidet numerische und alphanumerische Codes. Bei numerischen Codes werden nur Zahlenwerte kodiert, bei alphanumerischen Codes Ziffern, Buchstaben und Sonderzeichen.

Definition (Numerischer Code):

Ein Code, dessen Urmenge nur aus Zahlenwerten besteht heißt numerischer Code.

Bei numerischen Codes werden wiederum Wortcodes und Zifferncodes unterschieden: Beim Zifferncode gibt es eine Kodierungsvorschrift für die 10 Dezimalziffern.

Wortcodes:**Definition (Wortcode):**

Ein Code, der Zahlenwertigkeiten (und nicht einzelne Ziffern) codiert, heißt Wortcode.

Beispiel: 4-stelliger Dualcode für die Zahlen 0 – 15:

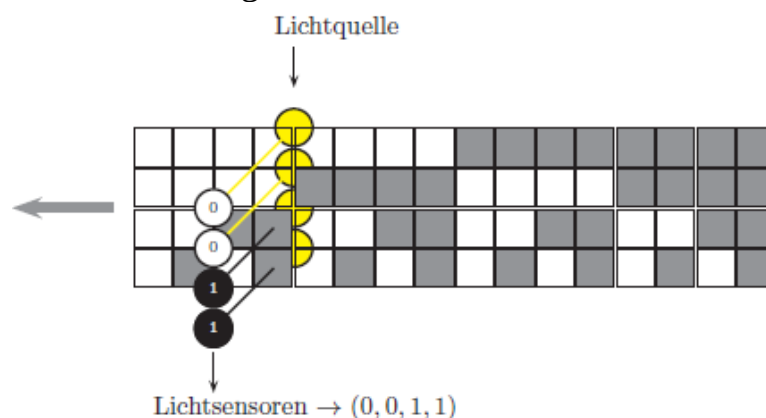
Urmenge	Bildmenge	Urmenge	Bildmenge
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

Bei der Zahl 12 werden nicht die Ziffern 1 und 2 separat codiert, sondern der Wert der gesamten Zahl 12.

Jedes der 16 Codewörter hat eine Länge von 4. Es handelt sich also (andere Kategorisierung!) um einen Blockcode.

Der Dualcode hat das Problem, dass sich u.U. mehrere Bits ändern während der numerische Wert nur um 1 weitergezählt wird. Das hat für bestimmte Anwendungen gravierende Nachteile:

Beispiel: Positionsbestimmung mittels Code-Lineal



An den Stellen, an denen sich mehrere Bits ändern, können während des Umschaltvorgangs ungültige Zwischenwerte auftreten. Dies würde ein Code vermeiden, bei dem sich bei aufeinander folgenden Codewörtern nur eine Stelle ändert (ein einschrittiger Code).

Definition (Hamming-Distanz):

Die Hamming-Distanz zweier Codewörter (eines Blockcodes) ist die Anzahl der Stellen, die sich unterscheiden.

Beispiel:

Die Hamming-Distanz der Codewörter (0111) und (1111) beträgt 1, die Hamming-Distanz der Codewörter (0111) und (1000) beträgt 4.

Definition (Stetiger Code):

Ein Code heißt stetig, wenn die Codewörter zwei aufeinanderfolgender Werte der Urmenge immer dieselbe Hamming-Distanz haben.

Definition (Einschrittigkeit):

Ein stetiger Code mit der Hamming-Distanz 1 heißt einschrittiger Code.

Ein Beispiel für einen einschrittigen Code ist der Gray-Code:

Zahl	Binär-Code	Gray-Code	Zahl	Binär-Code	Gray-Code
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000
8	1000	1100	0	0000	0000

Gegenüberstellung Binär-Code und Gray-Code mit Hamming-Distanz

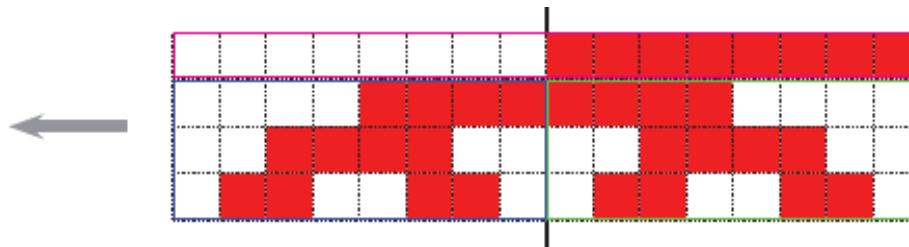
Die Konstruktion des Gray-Codes erfolgt folgendermaßen:

1. Hochzählen wie im Binärcode.
2. Beim Hinzunehmen der nächsten werden die bisher gesetzten Stellen horizontal gespiegelt.
3. Schritt 2 wiederholen bis alle Codewörter erzeugt wurden.

0	0	0	0
1	0	0	1
2	0	1	1
3	0	1	0
4	1	1	0
5	1	1	1
6	1	0	1
7	1	0	0

Stelle

Aufgrund des Bildungsgesetzes spricht man auch vom binären Reflex Code (BRC). Die folgende Darstellung zeigt einen 4-stelligen Graycode für die Anwendung als Codelineal:



Zifferncodes:

Zifferncodes werden auch als BCD-Codes bezeichnet (BCD := Binary Coded Decimal, d. h. binär codierte Dezimalziffer). Auch für alphanumerische Codes (siehe weiter unten) ist diese Bezeichnung gebräuchlich.

Definition (Zifferncode):

Ein Code, dessen Urmenge die Ziffern 0 . . . 9 sind heißt Zifferncode.

Die folgende Tabelle zeigt zwei Beispiele für BCD-Codes:

Ziffer	8-4-2-1 Code	4-2-2-1 Code
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 0
3	0 0 1 1	0 0 1 1
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	1 0 1 0
7	0 1 1 1	1 0 1 1
8	1 0 0 0	1 1 1 0
9	1 0 0 1	1 1 1 1

Es bleiben jeweils 6 Codewörter übrig, die nicht verwendet werden (die sogenannte **Pseudo-Tetraden**). Diese sind in der folgenden Tabelle aufgelistet:

8-4-2-1 Code	4-2-2-1 Code
1 0 1 0 = 10	0 1 0 0 = 2
1 0 1 1 = 11	0 1 0 1 = 3
1 1 0 0 = 12	1 0 0 0 = 4
1 1 0 1 = 13	1 0 0 1 = 5
1 1 1 0 = 14	1 1 0 0 = 6
1 1 1 1 = 15	1 1 0 1 = 7

Beide Codes sind bewertbar. Ein bewertbarer Code ist wie folgt definiert:

Definition (Bewertbarer Code):

Ein Zifferncode heißt bewertetbar, wenn sich für jede Stelle eine Wertigkeit angeben lässt, die für alle Codewörter gilt.

Die Stellenwertigkeit kann dabei auch negativ sein. Der folgende Code (Stibitz-Code, auch 3-Excess-Code genannt) ist nicht bewertetbar:

Ziffer	w_4	w_3	w_2	w_1
0	0	0	1	1
1	0	1	0	0
2	0	1	0	1
3	0	1	1	0
4	0	1	1	1
5	1	0	0	0
6	1	0	0	1
7	1	0	1	0
8	1	0	1	1
9	1	1	0	0
	1	1	0	1
	1	1	1	0
	1	1	1	1
	0	0	0	0
	0	0	0	1
	0	0	1	0

Es lässt sich hier keine Zuordnung für die Wertigkeiten von w_1 bis w_4 finden, die für alle 10 Codewörter gültig wäre.

Alphanumerische Codes:**Definition (Alphanumerischer Code):**

Ein alphanumerischer Code ist ein Code, dessen Urmenge sowohl Ziffern als auch Buchstaben und Sonderzeichen enthält.

Beispiel 1: Standard BCD-Code (Binary Coded Decimal Code).

Mit diesem Code können $2^6 = 64$ Zeichen kodiert werden.

Der Standard BCD-Code wurde zum EBCDI-Code (Extended BCD Interchange Code) weiterentwickelt (8 Bit).

			n_6	0	0	1	1
			n_5	0	1	0	1
n_4	n_3	n_2	n_1				
0	0	0	0	[ZW]		–	+
0	0	0	1	1	/	J	A
0	0	1	0	2	S	K	B
0	0	1	1	3	T	L	C
0	1	0	0	4	U	M	D
0	1	0	1	5	V	N	E
0	1	1	0	6	W	O	F
0	1	1	1	7	X	P	G
1	0	0	0	8	Y	Q	H
1	0	0	1	9	Z	R	I
1	0	1	0	0			
1	0	1	1	=	,	*	.
1	1	0	0	,	(		)
1	1	0	1				
1	1	1	0				
1	1	1	1				

Eine größere Bedeutung kommt allerdings dem ASCII-Code zu. Die Abkürzung steht für "American Standard Code for Information Interchange". Der ASCII-Code ist auf 7 Bit genormt (ISO 646, DIN 66 003).

Beispiel 2: ASCII-Code.

			n_7	0	0	0	0	1	1	1	1
			n_6	0	0	1	1	0	0	1	1
			n_5	0	1	0	1	0	1	0	1
n_4	n_3	n_2	n_1								
0	0	0	0	NUL	DLE	SP	0	@	P	'	p
0	0	0	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	STX	DC2	"	2	B	R	b	r
0	0	1	1	ETX	DC3	#	3	C	S	c	s
0	1	0	0	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	ACK	SYN	&	6	F	V	f	v
0	1	1	1	BEL	ETB	,	7	G	W	g	w
1	0	0	0	BS	CAN	(	8	H	X	h	x
1	0	0	1	HT	EM	)	9	I	Y	i	y
1	0	1	0	LF	SUB	*	:	J	Z	j	z
1	0	1	1	VT	ESC	+	;	K	[	k	{
1	1	0	0	FF	FS	,	<	L	\	l	
1	1	0	1	CR	GS	-	=	M	]	m	}
1	1	1	0	SD	RS	.	>	N	^	n	~
1	1	1	1	SI	US	/	1	O	-	o	DEL

Die kodierten Steuerzeichen haben die folgende Bedeutung:

ACK	Positive Rückmeldung (Acknowledge),
BEL	Klingel (Bell),
BS	Rückwärtsschritt (Backspace),
CAN	Ungültig (Cancel),
CR	Wagenrücklauf (Carriage Return),
DC	Gerätesteuerzeichen (Device Control Characters),
DEL	Löschen (Delete),
DLE	Datenübertragungsumschaltung (Data Link Escape),
EM	Ende der Aufzeichnung (End of Medium),
ENQ	Stationsaufforderung (Enquiry),
EOT	Ende der Übertragung (End of Transmission),
ESC	Code-Umschaltung (Escape),
ETB	Ende des Datenübertragungsblocks (End of Transmission Block),
ETX	Ende des Textes (End of Text),
FF	Formularvorschub (Form Feed),

FS	Hauptgruppen-Trennzeichen (File Separator),
GS	Gruppen-Trennzeichen (Group Separator),
HT	Horizontal-Tabulator (Horizontal Tabulation),
LF	Zeilenvorschub (Line Feed),
NAK	Negative Rückmeldung (Negative Acknowledge),
NUL	Füllzeichen (Null),
RS	Untergruppen-Trennzeichen (Record Separator),
SI	Rückschaltung (Shift-in),
SO	Dauerumschaltung (Shift-out)._____
SOH	Anfang des Kopfes (Start of Heading),
SP	Zwischenraum (Space),
STX	Anfang des Textes (Start of Text),
SUB	Substitutionszeichen (Substitute Character),
SYN	Synchronisierung (Synchronous Idle),
US	Teilgruppen-Trennzeichen (Unit Separator),
VT	Vertikal-Tabulator (Vertical Tabulation).

Dass der Zeichenvorrat der bisher behandelten Codes sehr begrenzt ist, zeigt ein kurzer Blick auf die wiedergegebenen Code-Tabellen. Fremdsprachige Zeichensätze können nur zu einem sehr kleinen Teil berücksichtigt werden; um auch Zeichen z.B. der chinesischen oder indischen Sprachen einzubauen, ist eine nennenswerte Erweiterung des Codes unumgänglich.

Der UNICODE verfügt zur Zeichenkodierung über 16 Bits, wodurch der Zeichenvorrat von 256 auf 65.536 Zeichen erweitert wird. Drei Eigenschaften des UNICODE sind wichtig im Vergleich mit anderen Zeichencodes:

- a) Es besteht eine direkte Kompatibilität mit dem sehr verbreiteten ASCII-Code, der eine Untermenge des UNICODE bildet; bei Reduzierung auf 8 Bits geht der UNICODE in den ASCII-Code über.
- b) Ein zweiter großer Bereich existiert, der kompatibel mit anderen Codes ist (Kompatibilitätsbereich).
- c) Der UNICODE lässt Raum für Erweiterungen (z.B. für länderspezifische oder zukünftige Zeichendefinitionen); ca. 25.000 Zeichen stehen dafür zur Verfügung.

Das letzte Beispiel kommt aus den Anfängen der Computertechnik: Es handelt sich um den 5-Bit Fernschreibcode, der mit Umschaltung zwischen Buchstaben- und Ziffernblocks arbeitet und somit eigentlich kein Code gemäß der obigen mathematischen Definition (eindeutige Abbildung) ist:

Beispiel 3: 5-Bit Fernschreibcode

Nr.	Bu	Zi	1	2	T	3	4	5	Nr.	Bu	Zi	1	2	T	3	4	5
1	A	-	•	•	○				17	Q	1	•	•	○	•		•
2	B	?	•		○		•	•	18	R	4		•	○		•	
3	C	:		•	○	•	•		19	S	'	•		○	•		
4	D	-F1-	•		○		•		20	T	5			○			•
5	E	3	•		○				21	U	7	•	•	○	•		
6	F	-F2-	•		○	•	•		22	V	=		•	○	•	•	•
7	G	-F2-		•	○		•	•	23	W	2	•	•	○			•
8	H	-F2-			○	•		•	24	X	/	•		○	•	•	•
9	I	8		•	○	•			25	Y	6	•		○	•		•
10	J	-F3-	•	•	○		•		26	Z	+	•		○			•
11	K	(	•	•	○	•	•		27	-F4-				○		•	
12	L	)		•	○			•	28	-F5-			•	○			
13	M	.			○	•	•	•	29	-F6-		•	•	○	•	•	•
14	N	,			○	•	•		30	-F7-		•	•	○		•	•
15	O	9			○		•	•	31	-F8-				○	•		
16	P	0	•	○	•		•		32	-F9-				○			

-F1- wer da?
 -F2- (nicht belegt)
 -F3- Klingel
 -F4- Wagenrücklauf
 -F5- Zeilenvorschub

-F6- BU = Buchstabenumschaltung
 -F7- Zi = Ziffernumschaltung
 -F8- Zwischenraum
 -F9- (nicht benutzt)

2.4 Digitale Schaltkreise

Schaltkreise, die einem oder mehreren Binärsignalen am Eingang ein Binärsignal am Ausgang zuweisen, bezeichnet man als Gatter (engl. Gates). Da bei der Verarbeitung der Signale physikalische Vorgänge ablaufen, ist die Verarbeitungsgeschwindigkeit endlich; das Ausgangssignal steht also erst nach einer Verzögerungszeit zur Verfügung. Diese Verzögerungszeit bezeichnet man als Gatterlaufzeit (engl. propagation delay time).

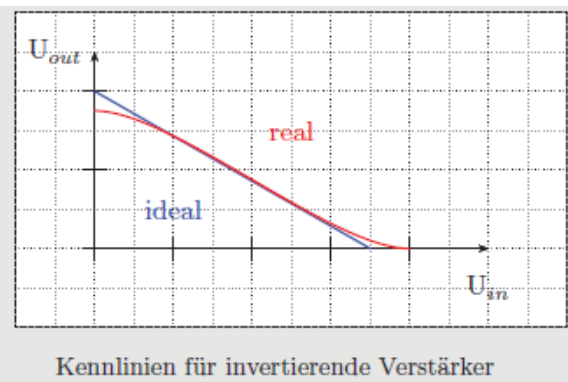
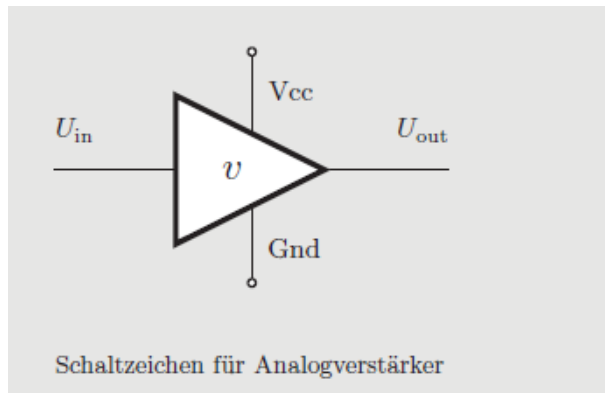
Die Gatterlaufzeit ist abhängig von der eingesetzten Technologie, der Temperatur, den anliegenden Spannungspegeln und weiteren elektrischen Parametern und liegt typischerweise im Bereich von 100 ps bis 100 ns. Die Gatterlaufzeit kann im Folgenden noch weitgehend vernachlässigt werden.

Schaltkreise mit einem Eingangssignal

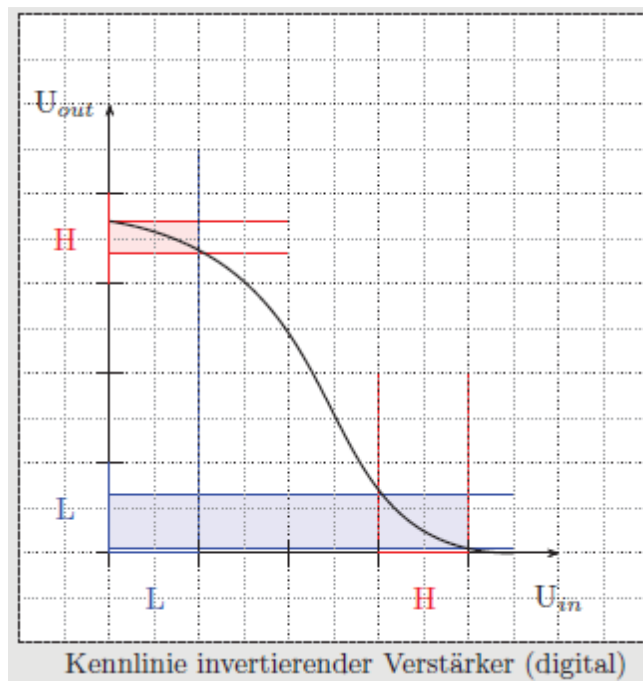
In der Analogtechnik wird ein möglichst lineares Verhalten von Verstärkern angestrebt. Für den idealen Verstärker gilt:

$$U_{out} = v \cdot U_{in} + U_{off}$$

In der Realität sind Verstärker nicht zu 100% ideal (vgl. Abbildung).



In der Digitaltechnik können unterschiedliche Spannungen die selbe Information bedeuten (vgl. Kapitel 1). Daher kommen i.a. Elemente mit nichtlinearen Kennlinien zum Einsatz. Bei solchen nichtlinearen Kennlinien können Schwankungen der Eingangsspannung kompensiert werden (Signalverbesserung).



(H = High: Spannung mit hohem Pegel; L = Low: Spannung mit niederem Pegel)

Für die Beschreibung der Funktion genügt die so genannte Pegeltabelle:

U_{in}	U_{out}
H	L
L	H

Ein nicht-invertierender Verstärker hat dann die folgende Pegeltabelle:

U_{in}	U_{out}
L	L
H	H

Der nicht-invertierende Verstärker, der an dem logischen Wert nichts ändert, wird in der Digitaltechnik auch als Treiber oder Puffer (engl. Buffer) bezeichnet. Er dient der Signalverstärkung („fan-out“).

Schaltsymbole:

Die Darstellung der Schaltsymbole für Treiber und Inverter sind genormt. Dabei haben sich verschiedene Darstellungsweisen etabliert. Im Rahmen dieses Skriptes werden die Symbole der IEC 60617 verwendet. Dies gilt auch für alle im Folgenden aufgeführten Symbole.

Name	Schalttabelle	Internationale Norm IEC 60617	Alte deutsche Norm DIN 40700	Amerikanische Norm ANSI 91-1984						
Inverter	<table><tr><th>U_{in}</th><th>U_{out}</th></tr><tr><td>L</td><td>H</td></tr><tr><td>H</td><td>L</td></tr></table>	U_{in}	U_{out}	L	H	H	L			
U_{in}	U_{out}									
L	H									
H	L									
Treiber, Puffer	<table><tr><th>U_{in}</th><th>U_{out}</th></tr><tr><td>L</td><td>L</td></tr><tr><td>H</td><td>H</td></tr></table>	U_{in}	U_{out}	L	L	H	H			
U_{in}	U_{out}									
L	L									
H	H									

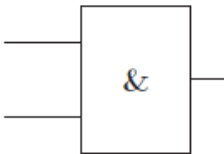
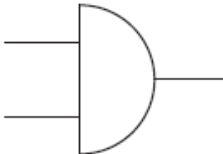
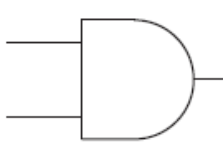
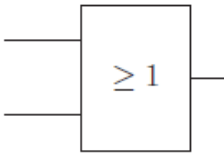
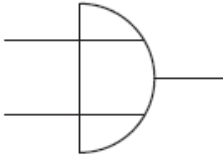
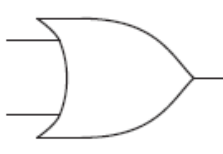
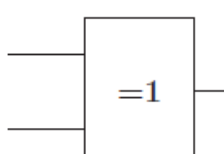
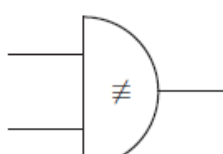
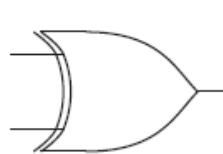
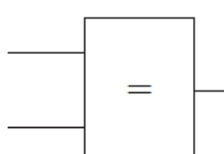
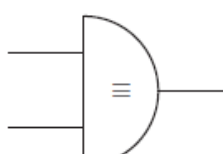
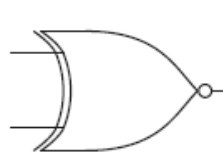
Schaltkreise mit zwei Eingangssignalen

Die Funktion von Schaltkreisen mit zwei Eingängen kann ebenfalls mittels Pegeltabellen beschreiben werden. Diese besitzt dann 4 Zeilen.

Die wichtigsten Schaltkreise mit zwei Eingangssignalen sind das UND- und das ODER-Gatter. Diese realisieren die zwei-stelligen Booleschen Grundverknüpfungen „Konjunktion“ und „Disjunktion“ (siehe Kap. 3).

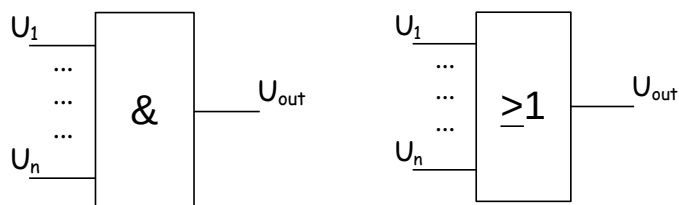
Weitere Schaltkreise sind das Antivalenzgatter (XOR-Gatter) und das Äquivalenzgatter (XNOR-Gatter).

In der folgenden Tabelle sind jeweils Pegeltabelle (Schalttabelle) und Symbole angegeben:

Name	Schalttabelle	Internationale Norm IEC 60617	Alte deutsche Norm DIN 40700	Amerikanische Norm ANSI 91-1984															
UND-Gatter (AND)	<table><tr><th>U_1</th><th>U_2</th><th>U_{out}</th></tr><tr><td>L</td><td>L</td><td>L</td></tr><tr><td>L</td><td>H</td><td>L</td></tr><tr><td>H</td><td>L</td><td>L</td></tr><tr><td>H</td><td>H</td><td>H</td></tr></table>	U_1	U_2	U_{out}	L	L	L	L	H	L	H	L	L	H	H	H			
U_1	U_2	U_{out}																	
L	L	L																	
L	H	L																	
H	L	L																	
H	H	H																	
ODER-Gatter (OR)	<table><tr><th>U_1</th><th>U_2</th><th>U_{out}</th></tr><tr><td>L</td><td>L</td><td>L</td></tr><tr><td>L</td><td>H</td><td>H</td></tr><tr><td>H</td><td>L</td><td>H</td></tr><tr><td>H</td><td>H</td><td>H</td></tr></table>	U_1	U_2	U_{out}	L	L	L	L	H	H	H	L	H	H	H	H			
U_1	U_2	U_{out}																	
L	L	L																	
L	H	H																	
H	L	H																	
H	H	H																	
Antivalenz-Gatter (XOR) Exklusiv-Oder	<table><tr><th>U_1</th><th>U_2</th><th>U_{out}</th></tr><tr><td>L</td><td>L</td><td>L</td></tr><tr><td>L</td><td>H</td><td>H</td></tr><tr><td>H</td><td>L</td><td>H</td></tr><tr><td>H</td><td>H</td><td>L</td></tr></table>	U_1	U_2	U_{out}	L	L	L	L	H	H	H	L	H	H	H	L			
U_1	U_2	U_{out}																	
L	L	L																	
L	H	H																	
H	L	H																	
H	H	L																	
Äquivalenz-Gatter (XNOR)	<table><tr><th>U_1</th><th>U_2</th><th>U_{out}</th></tr><tr><td>L</td><td>L</td><td>H</td></tr><tr><td>L</td><td>H</td><td>L</td></tr><tr><td>H</td><td>L</td><td>L</td></tr><tr><td>H</td><td>H</td><td>H</td></tr></table>	U_1	U_2	U_{out}	L	L	H	L	H	L	H	L	L	H	H	H			
U_1	U_2	U_{out}																	
L	L	H																	
L	H	L																	
H	L	L																	
H	H	H																	

Schaltkreise mit mehr als zwei Eingangssignalen

Betrachtet werden in diesem Abschnitt UND- bzw. ODER-Gatter für beliebig viele Eingänge:



UND-Gatter:

Der Ausgang besitzt den Pegel H, wenn alle Eingänge den Pegel H aufweisen. (Ein L-Pegel am Eingang erzwingt den L-Pegel am Ausgang).

ODER-Gatter:

Der Ausgang besitzt den Pegel L, wenn alle Eingänge den Pegel L aufweisen. (Ein H-Pegel am Eingang erzwingt den H-Pegel am Ausgang).

Die nachfolgende Tabelle zeigt die Pegeltabelle eines UND-Gatters bzw. ODER-Gatters mit 3 Eingängen.

Eingänge			UND	ODER
L	L	L	L	L
L	L	H	L	H
L	H	L	L	H
L	H	H	L	H
H	L	L	L	H
H	L	H	L	H
H	H	L	L	H
H	H	H	H	H